

COMP101

Introduction to Programming

2025-26

Week-01 Tutorial-01

Python Support

No Supervision

Reading only

Python IDLE Environment

Python is downloadable from the official Python site

<https://www.python.org/downloads/>

Lot of support sites for how to use it, including:

<https://www.tutorialsteacher.com/python/python-idle>

<https://realpython.com/python-idle/>

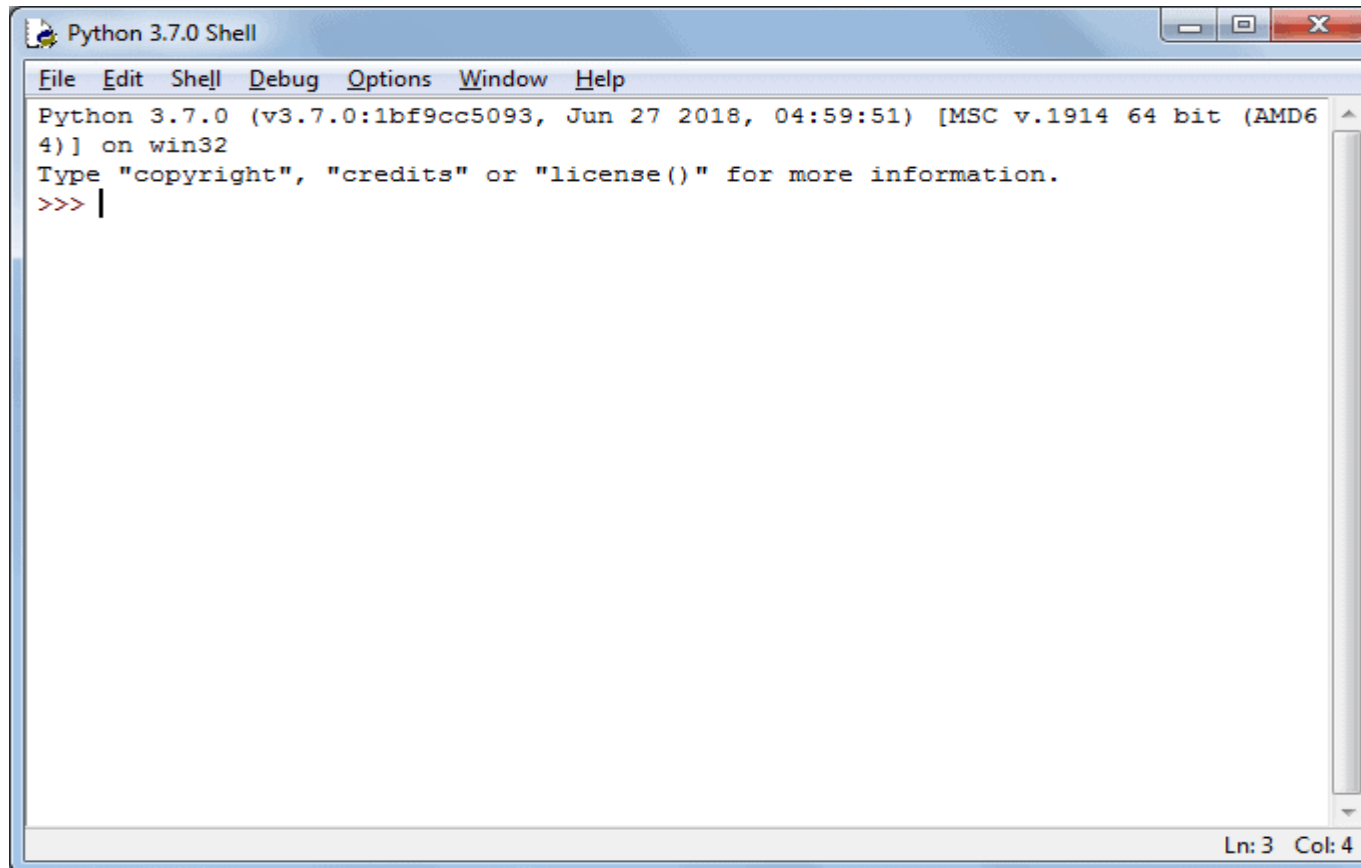
Note: Module is 'Introduction to Programming', not 'Introduction to Python' – we are using Python to learn how to program

Python IDLE Environment

- IDLE (Integrated Development and Learning Environment)
 - Contains a shell window ('Python Shell') and a script editor
- Shell window opens as the default screen
 - Python is an interpreted language – it executes code ('statement') one line at a time
 - This can be seen using the Python Shell
- Script editor provides editing functions to type in multiple statements
 - It allows us to save these statements as a file (a 'Python script')
 - We get a .py file which is executable

Python IDLE Environment

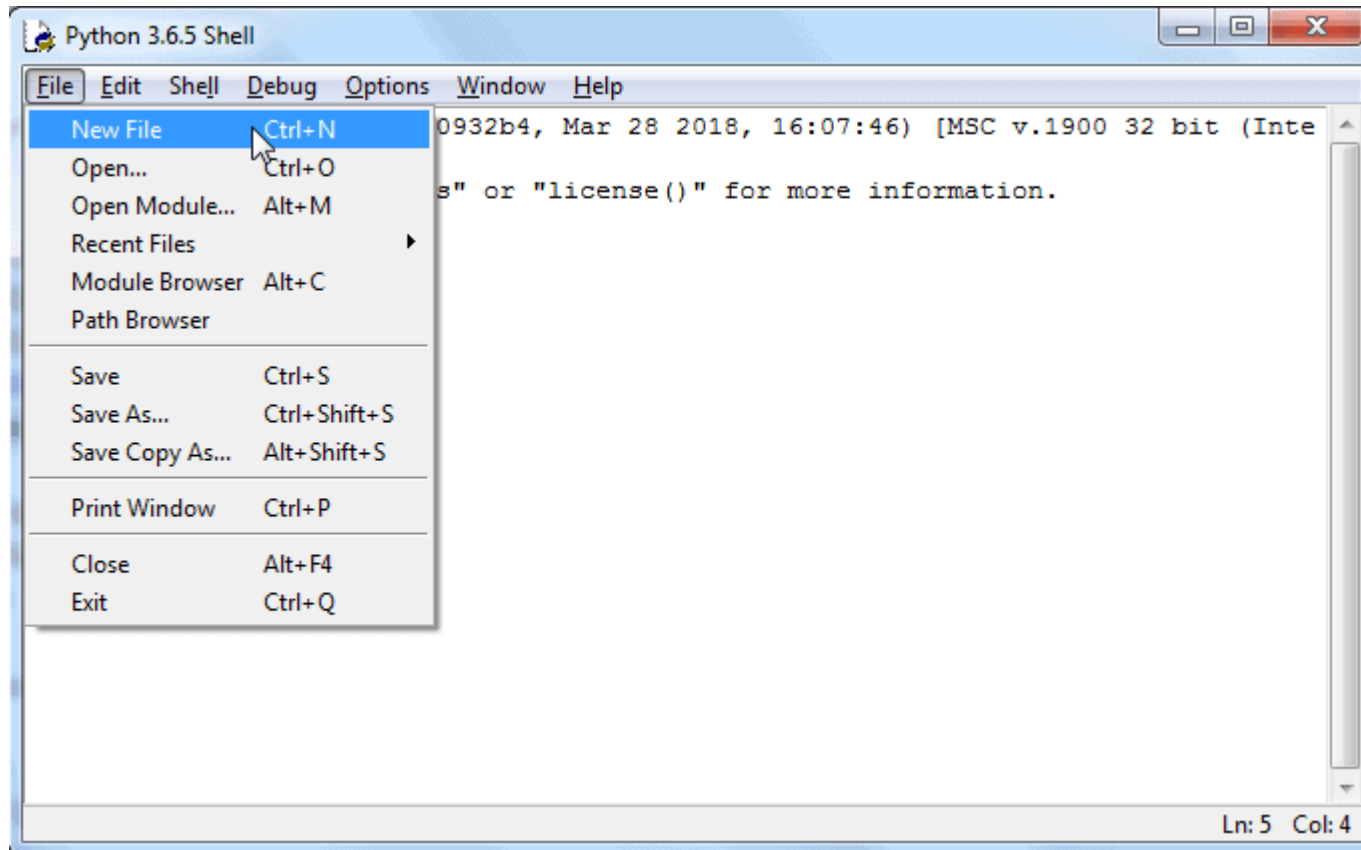
- Python Shell Window – first thing to be seen on screen



At the command prompt (>>>) we can type in a statement and it executes when we press the return key. Python is an interpreted language – it executes one statement at a time. The shell window is the 'interpreter window'

Python IDLE Environment

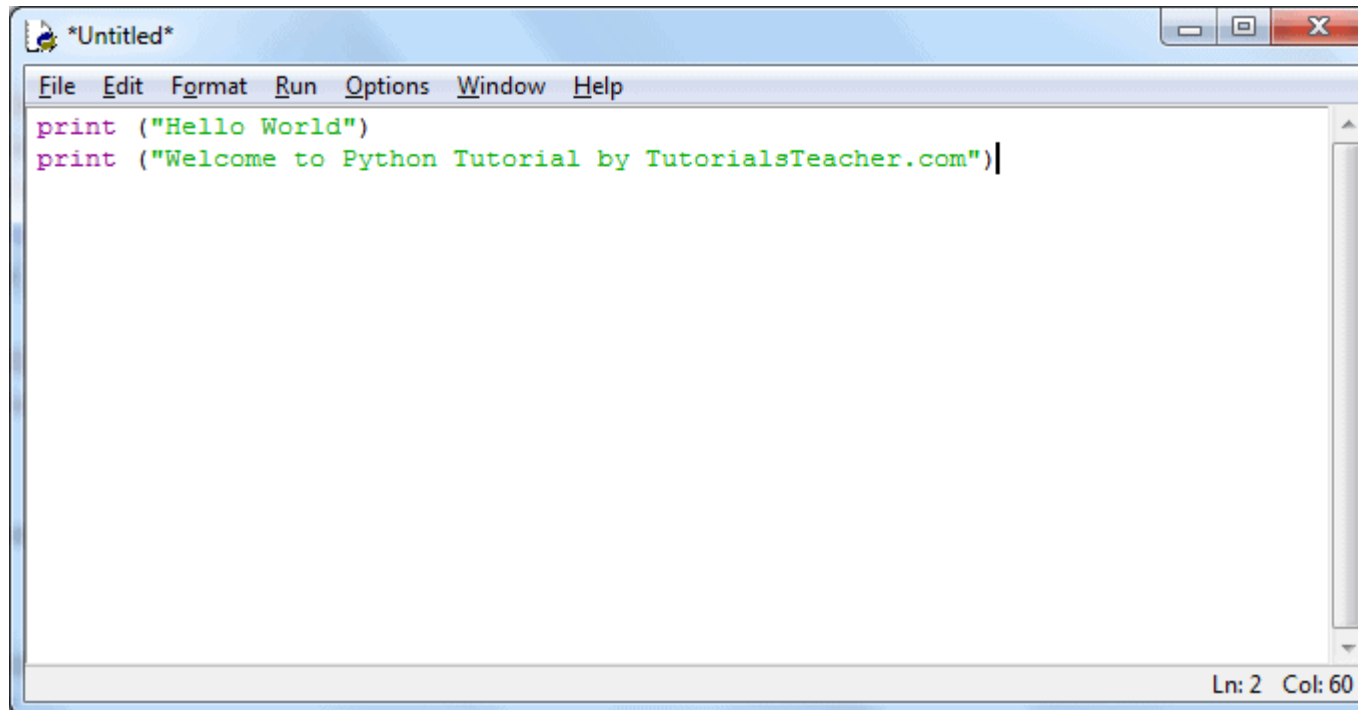
To create a .py file from the shell, we need the script editor



Python IDLE Environment

In the script editor we type in our code (program)

Save it when finished (default filetype is .py so don't change it)



Save it with:

File – save/save as:

Use any name to save it –
but keep it sensible and
keep it as .py

Run it (execute it) with:

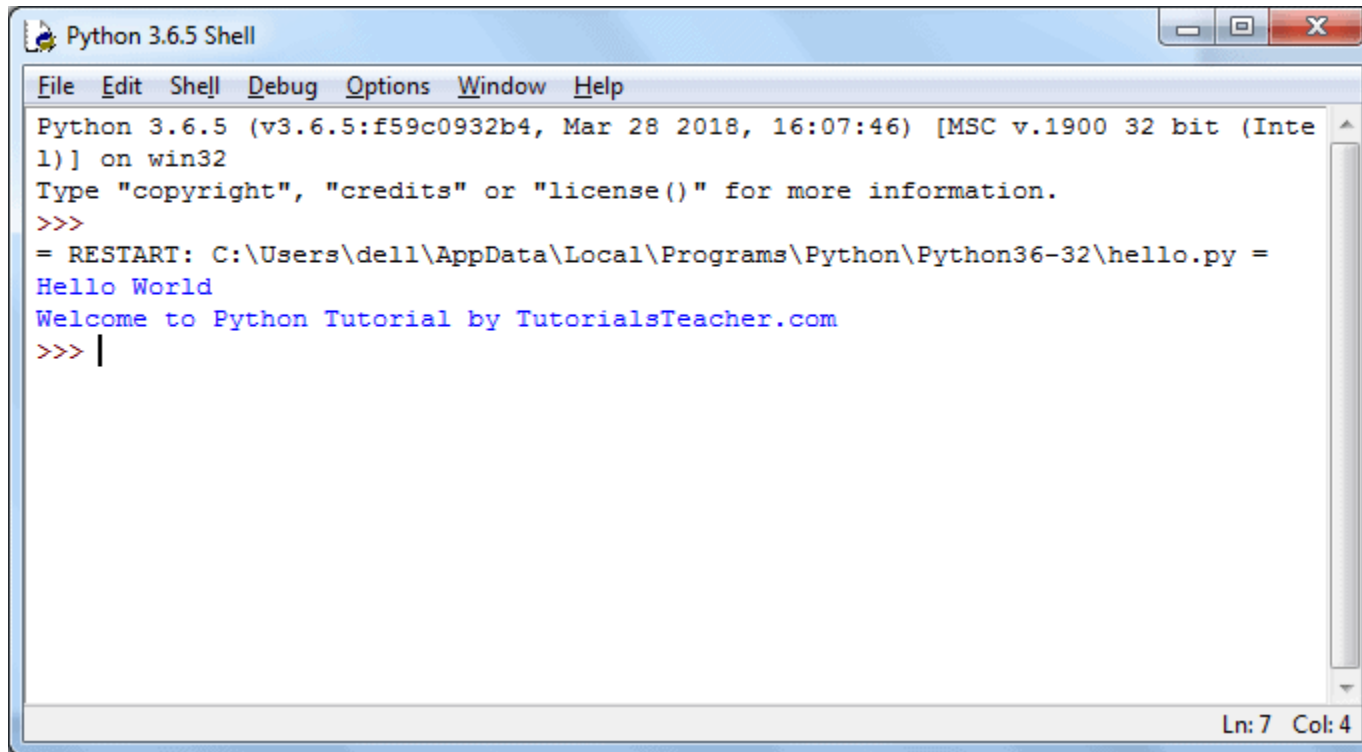
F5 or

Run – Run Module

The output appears in the
shell window

Python IDLE Environment

On execution of a Python script as a .py file output appears in shell



The screenshot shows a window titled "Python 3.6.5 Shell" with a menu bar (File, Edit, Shell, Debug, Options, Window, Help). The text inside the window is as follows:

```
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:\Users\dell\AppData\Local\Programs\Python\Python36-32\hello.py =
Hello World
Welcome to Python Tutorial by TutorialsTeacher.com
>>> |
```

The status bar at the bottom right indicates "Ln: 7 Col: 4".

Why Python and not Java to learn programming

- Python is good for learning and solving real problems
- Python is listed as an O-O language but is very good procedurally
 - This is because functions are implemented as methods of object
 - Especially good for learning programming without the overheads of OO
- OO programming has a lot of overheads
 - Even simple programs such as “Hello World” need over ten lines of code
 - Learning to program in O-O means learning to program and learning the OO environment at the same time – can get tricky
 - OO paradigms are not always good at supporting the procedural approach

Why Procedural Paradigm

- Small procedural programs (<1000 lines of code) perform better with procedural paradigm
 - Good if the state of the program does not change much
 - OO isn't always a clearer way to program problems as it can create un-needed complexity
 - The complexity of OO languages can involve writing many lines of code to support the OO environment before writing code to solve the problem
- But large procedural programs can become complex
 - This is because maintenance and re-use is critical for programming efficiency
 - OO is better at larger programs where change is common as code can be modularised

Why Object Oriented Paradigm

- OO was developed for managing the complexity of problem domains where change is common and we can exploit inheritance etc to increase programming efficiency
- Instead of complex data structures and many functions to operate on each structure as in procedural paradigms, OO couples data structures with functionality into an object
- Objects are identified by class and hierarchies of classes (inheritance)
- Complex objects can have references to other objects (composition)

Python Support packages

Do not use in COMP101

- Python is downloadable from the official Python site (recommended for COMP101)
 - Comes with an inbuilt set of modules (held in 'packages') to support popular programming tasks
- Python has become very popular amongst the science community
 - Ease of programming and flexibility
 - Further modules can be downloaded separately
- Several platforms have Python in them plus extra packages for advanced programming
 - **COMP101 DOES NOT NEED THESE EXTRA PACKAGES IN THESE PLATFORMS**
 - Don't access any extra packages to avoid overload and getting lost in the environment – learn to program first
 - Most of these packages are for advanced projects, distributed systems, advanced science applications etc

Python Support packages

Do not use in COMP101

- There are six core packages (amongst others)
- NumPy n- dimensional array handling and analysis
- SciPy general maths/science/engineering environment
- Matplotlib 2-dimensional modelling/visualisation as plots, histograms, barcharts etc
- IPython Interactive Python - enhanced interactive mode for Python shell and kernel (IPython) – gave rise to Jupyter and subsequently Jupyter Notebook
- SymPy symbolic mathematics
- Pandas data structure and analysis

Python Support packages

Do not use in COMP101

- Other packages
- PyPy faster alternative to Python – DON'T US IT
- PyPi Python Package Index – helps users store and distribute their own packages
- pip Python Indexing Project – helps to install 'packages' which hold all the files needed for a 'module'. A module contains the code libraries to make the module run

Python Support packages

Jupyter Notebook – Do not use in COMP101

- Jupyter Notebook is the successor to IPython Notebook
- Jupyter Notebook includes Python 3.x
- Blended learning, blogs, open-source web apps, create/share documents of live code, equations, visualisations, numerical simulation, statistical modelling, machine learning, data analytics, science tools, . . .

Python Support packages

Anaconda – Do not use in COMP101

- Python/R open-source data science and machine learning platform
- Contains approx 1500 packages
- v.2019.03 (now uses versioning format as year.month)
- Includes Python 3.x, Jupyter Notebook, NumPy, SciPy, Matplotlib, IPython, SymPy, Pandas
- Includes scikit learn (machine and deep learning models, TensorFlow, Theano), Conda (v4.6.11 to manage libraries, dependencies, environments, package management system)
- Miniconda is the lightweight version

Python Support packages

Anaconda – Do not use in COMP101

- Includes
- Visualisation Packages
 - Matplotlib, Bokeh, Datashader, Holoviews
- Data Analysis Packages
 - Numpy, Pandas, Dask, Numba
- Other packages
 - MKL, PyTables, OpenSSL, Cryptography, . . .
- Pycharm
 - Professional Python IDE
 - Development of AnacondaCON
 - In partnership with JetBrains to integrate Anaconda Distribution with Pycharm